

maplab: An Open Framework for Research in Visual-inertial Mapping and Localization

Thomas Schneider^{*†}, Marcin Dymczyk^{*†}, Marius Fehr^{*†},
 Kevin Egger[†], Simon Lynen^{†§}, Igor Gilitschenski[‡], Roland Siegwart[†]
[†]Autonomous Systems Lab, ETH Zürich, [‡]CSAIL, MIT, [§]Google Inc., Zürich
^{*}contributed equally

Abstract—Robust and accurate visual-inertial estimation is crucial to many of today’s challenges in robotics. Being able to localize against a prior map and obtain accurate and drift-free pose estimates can push the applicability of such systems even further. Most of the currently available solutions, however, either focus on a single session use-case, lack localization capabilities or an end-to-end pipeline. We believe that only a complete system, combining state-of-the-art algorithms, scalable multi-session mapping tools, and a flexible user interface, can become an efficient research platform.

We therefore present maplab, an open, research-oriented visual-inertial mapping framework for processing and manipulating multi-session maps, written in C++. On the one hand, maplab can be seen as a ready-to-use visual-inertial mapping and localization system. On the other hand, maplab provides the research community with a collection of multi-session mapping tools that include map merging, visual-inertial batch optimization, and loop closure. Furthermore, it includes an online frontend that can create visual-inertial maps and also track a global drift-free pose within a localization map. In this paper, we present the system architecture, five use-cases, and evaluations of the system on public datasets. The source code of maplab is freely available for the benefit of the robotics research community.

I. INTRODUCTION

The ever growing deployment of simultaneous localization and mapping (SLAM) systems poses novel challenges for the robotics community. Availability of precise, drift-free pose estimates both outdoors and indoors has become a vital requirement of numerous robotics applications, such as navigation or manipulation. The increasing popularity of visual-inertial estimation systems created a strong incentive to improve their robustness to viewpoint and appearance changes (daylight, weather, seasons, etc.) or rapid motion. Current research efforts aim to collect data using heterogeneous agents, build maps of larger scale, cover various visual appearance conditions and maintain maps over a long time horizon. Investigating these and many related challenges requires a multi-session end-to-end mapping system that can be easily deployed on various robotic platforms and provides ready-to-use algorithms with state-of-the-art performance. At the same time it needs to offer high flexibility necessary for conducting research.

Most openly available frameworks for visual and visual-inertial SLAM either focus on a single-session case [1] or only provide large-scale batch optimization without an online frontend [2]. Usually, they are crafted for a very specific pipeline without a separation between the map structure and

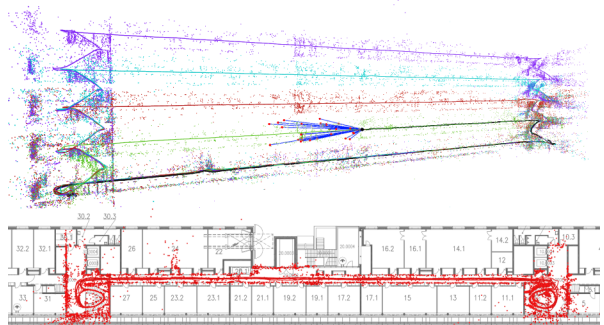


Fig. 1: The maplab framework can build consistent visual-inertial maps from multiple mapping sessions. Here, 4 separate sessions are merged and jointly refined. The global map can then be used by odometry and localization frontend to correct for any drift when revisiting the area. The floorplan is overlaid with the landmarks of all floors demonstrating the accuracy and consistency of the map alignment.

algorithms. They often lack completeness and will not offer a full workflow such that a map can be created, manipulated, merged with previous sessions and reused in the frontend within a single framework. This impairs the flexibility of such systems, a key for rapid development and research.

This work addresses this problem by introducing maplab¹, an open visual-inertial mapping framework, written in C++. In contrast to existing visual-inertial SLAM systems, maplab does not only provide tools to create and localize from visual-inertial maps but also provides map maintenance and processing capabilities. These capabilities are offered as a set of tools accessible in a convenient console that can easily be extended through a plugin system. These tools involve multi-session merging, sparsification, loop closing, dense reconstruction and visualization of maps. Additionally, maplab includes ROVIOLI (ROVIO with Localization Integration), a mapping and localization frontend based on ROVIO [3], a patch-based visual-inertial odometry system.

Maplab has been extensively field tested and has been deployed on a variety of robotic platforms including micro aerial vehicles [4], autonomous planes [5, 6], autonomous cars [7], autonomous underwater vehicles [8], and walking robots [9]. It has also served as a research platform for map summarization [10–13], map quality evaluation [14], multi-

¹Maplab is available at: www.github.com/ethz-asl/maplab

session 3d reconstruction [15], topological mapping [16], visual localization [17–19], and decentralized mapping [20].

To the best of our knowledge, maplab is the first visual-inertial mapping framework that integrates a wide variety of use-cases within a single system. Maplab is free, open-source, and has already proved to be of great use for various research and industry projects. We strongly believe that the robotics community will harness it both as an off-the-shelf mapping and localization solution, as well as a mapping research testbed. The contributions of this work can be summarized as follows:

- it introduces a general purpose visual-inertial mapping framework using feature-based maps with multi-session support;
- it introduces ROVIOLI, a robust visual-inertial estimator tightly coupled with a localization system;
- it presents examples of algorithms and data structures for modifying and maintaining maps including map merging, sparsification, place recognition, and visualization;
- it highlights the extensibility of the system that makes it well suited for research;
- it provides evaluation of selected components of the framework.

II. RELATED WORK

There are several openly available visual and visual-inertial SLAM systems. One of the earliest examples is PTAM [21], a lightweight approach for mapping and tracking a local map in parallel. It was originally developed for augmented reality applications so it offers neither large-scale localization nor any offline processing tools. More recent examples include OKVIS [1], a visual-inertial keyframe-based estimator. This approach tracks a local map built from recently acquired keyframes, which minimizes the drift locally. Similarly, semi-dense [22] and dense [23] odometry frameworks achieve high-quality pose estimates by using photometric error formulations instead of feature-based matching. None of these methods, however, supports global localization against a previously recorded map.

ORB-SLAM [24] and ORB-SLAM2 [25] are vision-based frameworks that offer the possibility to create a map of the environment and then reuse it in a consecutive session, which closely relates to the workflow we propose here. In contrast to these systems, maplab offers an offline processing toolkit centered around a console user interface, which guarantees high flexibility and permits users to add their own extensions or modify the processing pipelines. We consider the ability to merge multiple mapping sessions into a single, consistent map and to refine it using a visual-inertial least-squares optimization a core capability of maplab that differentiates it from ORB-SLAM. Another difference worth emphasizing is the online frontend of maplab, ROVIOLI. Using image intensity within patches instead of point features guarantees a high level of robustness, even in the presence of motion blur [3].

Incorporating the capability to process multiple maps has received considerable attention in the SLAM research community with [26] being one of the earliest works incorporating multiple maps in a hybrid metric-topological approach to multi-session mapping. Use of anchor nodes to stitch together posegraphs from multiple mapping sessions is proposed in [27]. Trying to establish topological associations between maps is also proposed in [28], where maps are stored as a set of experiences. In contrast, maplab stores a unified localization map allowing to use a carefully selected subset of features, e.g. based on the current appearance conditions [7].

Systems that aim to reconstruct the 3d structure from large collections of unordered images [2, 29, 30] also contain functionalities similar to maplab. They typically offer efficient implementations of large-scale bundle adjustment optimization and advanced image and feature matching techniques. They lack, however, algorithms that process inertial data and cannot be run directly on a robotic platform in order to provide pose estimates online.

III. THE MAPLAB FRAMEWORK

From the user perspective, the framework consists of two major parts:

- i. The online **VIO and localization frontend**, ROVIOLI, that takes raw visual-inertial sensor data. It outputs (global) pose estimates and can be used to build visual-inertial maps.
- ii. The (offline) **maplab-console** that lets the user apply various algorithms on maps in an offline batch fashion. It does also serve as a research testbed for new algorithms that operate on visual-inertial data.

The maplab framework follows an extensible and modular design. All software components are organized in packages, which are built using catkin, the official build system of ROS [31]. The C++11 standard is used throughout the framework and third-party dependencies are limited to popular and well-maintained libraries, among others Eigen [32] for linear algebra and Ceres [33] for non-linear optimization. Additionally, the framework provides ROS interfaces to conveniently input raw sensor data and output the results, such as pose estimates for an easy deployment on a robotic systems. The framework uses RViz as a 3d visualization tool to both visualize the state of the online mapping algorithms and the results of the offline processing from the maplab console.

A. Notation

Throughout this document and the source-code, we use the notation as defined in this section. A transformation matrix $\mathbf{T}_{AB} \in \text{SE}(3)$ takes a vector ${}_B\mathbf{p} \in \mathbb{R}^3$ from the frame of reference $\mathcal{F}_B$ to the frame of reference $\mathcal{F}_A$. It can be partitioned into a rotation matrix $\mathbf{R}_{AB} \in \text{SO}(3)$ and a translation vector ${}_A\mathbf{p}_{AB} \in \mathbb{R}^3$ as:

$$\begin{bmatrix} {}_A\mathbf{p} \\ 1 \end{bmatrix} = \mathbf{T}_{AB} \cdot \begin{bmatrix} {}_B\mathbf{p} \\ 1 \end{bmatrix} = \begin{bmatrix} \mathbf{R}_{AB} & {}_A\mathbf{p}_{AB} \\ \mathbf{0} & 1 \end{bmatrix} \cdot \begin{bmatrix} {}_B\mathbf{p} \\ 1 \end{bmatrix} \quad (1)$$

The operator $\mathbf{T}_{AB}(\cdot)$ is defined to transform a vector in $\mathbb{R}^3$ from $\mathcal{F}_B$ to the frame of reference $\mathcal{F}_A$ as $\mathbf{a}_P = \mathbf{T}_{AB}(\mathbf{b}_P)$ according to Eq. (1).

B. Workflow for multi-session mapping and localization

The typical workflow for a mapping and localization session within the maplab system is illustrated in Fig. 2. Often, it is beneficial to build a single localization map from multiple mapping sessions to ensure a good spatial and temporal (i.e. different appearances) coverage of the area. An initial, open loop map is built in each session using ROVIOLI in VIO mode and stored to disk. The maps can then be refined using various (offline) tools such as loop closure detection, visual-inertial optimization or co-registration of multiple sessions (map merging). Detailed inspection of the maps is possible using a large set of different visualizations, statistics and queries. More advanced modules allow, e.g., to create a dense representation (TSDF, occupancy, etc.) of the environment using data from a depth sensor or from stereo.

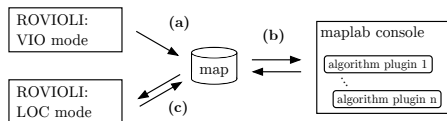


Fig. 2: Typical workflow in maplab: (a) In VIO mode, ROVIOLI estimates the pose of an agent w.r.t. a (drifting) local frame; additionally a map is built based on these estimates. (b) Resulting maps can be loaded in the maplab-console where all of the available algorithms can be applied, e.g. map alignment and merging, VI optimization, loop-closure. (c) In LOC mode, ROVIOLI can load the updated map to track a global (drift-free) pose online.

The resulting (multi-session) map can then be exported as a compact localization map and used by ROVIOLI (in LOC mode) for online localization during a second visit to the same place. Continuous online localization enables accurate tracking of a global pose w.r.t. a known 3d structure and thus compensates for drift in the visual-inertial state estimation.

C. maplab console: the offline user interface

The maplab framework uses a console user interface to manipulate maps offline. Multiple maps can be loaded into the console simultaneously, facilitating multi-session mapping experiments. All algorithms are available through console commands and can be applied to the loaded maps. Parameters specific to each algorithm are set by console flags or a flag file and can be modified at runtime. Combined with the real-time visualization of the map in RViz, this greatly facilitates algorithm prototyping and parameter tuning. It is possible to combine multiple algorithms and experiment with entire processing pipelines. Changes can be easily reverted by saving and reloading intermediate states of a map from disk.

The console uses a plugin architecture¹ and automatically detects all available plugins within the build workspace at run time. Therefore, the integration of a new algorithm or

¹For more details, tutorials and documentation, please visit our wiki page: www.github.com/ethz-asl/maplab/wiki

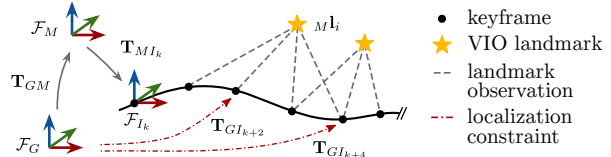


Fig. 3: Coordinate frames used in maplab and ROVIOLI: $\mathcal{F}_G$: global, gravity-aligned map frame; all missions are anchored in this frame. $\mathcal{F}_{M_k}$: gravity-aligned frame that represents the origin of a mission k equivalent to the origin of the VIO. $\mathcal{F}_{I_k}$: IMU frame at time stamp k (body frame).

functionality is possible without any changes to the core packages. For algorithms that operate on the standard visual-inertial map datatype (see Section III-D), no interfacing work will be necessary.

D. Map structure

The framework uses a data structure, called VI-map, for visual-inertial mapping data. The VI-map contains the raw measurements of all sensors and a sparse reconstruction of the covered environment. Each map may contain multiple *missions* where each is based on a single recording session. The core structure of a mission is a graph consisting of *vertices* and *edges*. A *vertex* corresponds to a state captured at a certain point in time. It contains a state estimate (pose $\mathbf{T}_{MI_k}$, IMU biases, velocity) and visual information from the (multi-)camera system including keypoints, descriptors (BRISK [34] or FREAK [35]), tracking information and images. An *edge* connects two neighboring vertices. While there are a few different types of edges in maplab, the most common type is the IMU edge. It contains the inertial measurements recorded between the vertices that the edge connects. Visual observations tracked by multiple vertices are triangulated as 3d landmarks. The landmark itself is stored within the vertex that first observed it. Loop-closures might link observations of one mission to a landmark stored in another mission.

Fig. 3 illustrates the map structure and introduces the relevant coordinate frames. Each mission is anchored in the global coordinate frame $\mathcal{F}_G$ using a transformation $\mathbf{T}_{GM_i}$. The poses $\mathbf{T}_{MI_j}$ of mission i are expressed w.r.t. the mission frame $\mathcal{F}_{M_i}$. Therefore, it suffices to manipulate the transformation $\mathbf{T}_{GM_i}$ to anchor multiple missions in a single global coordinate system without the need for updating any vertex poses or landmark positions.

The map structure can be serialized to the Google Protocol buffer format, enabling portable file serialization and network transmission. Furthermore, data-intensive objects (such as images, dense reconstructions, etc.) can be attached to the maps using a resource management system. Resources are linked to either a *vertex* or a set of *missions* or simply a timestamp, and are stored on the file system separate from the main mapping data. This architecture allows for (cached) loading such (potentially large) objects on demand, effectively reducing the peak memory usage. This facilitates research in areas such as dense reconstruction and image-based/enhanced localization on large-scale maps that might otherwise exhaust the available memory on certain platforms.

E. Core packages of maplab

The maplab framework incorporates implementations of several state-of-the-art algorithms. All of them are conveniently accessible from the maplab console. We only briefly highlight the ones that, in our opinion, bring a particular value to the robotics community:

VIWLS: visual-inertial weighted least-squares optimization with cost terms similar to [1]. The main batch optimization algorithm of the framework is used to refine maps e.g. after initialization with ROVIOLI or after loop-closures have been established. By default, the optimization problem is constructed using visual and inertial data, but optionally it can include wheel odometry, GPS measurements or other types of pose priors.

Loop closure/localization: a complete loop closure and localization system based on binary descriptors. The search backend uses an inverted multi-index for efficient nearest neighbor retrieval on projected binary descriptors. The algorithm is a (partial) implementation of [36].

ROVIOLI: online visual-inertial mapping and localization frontend, see Section III-F for details.

Posegraph relaxation: posegraph optimization using edges introduced by the loop closure system. The algorithm is similar to [37]. Optionally, a Cauchy loss might be used to increase the robustness against false loop closures.

aslam_cv2: a collection of computer vision data structures and algorithms. It includes various camera and distortion models as well as algorithms for feature detection, extraction, tracking and geometric vision.

Map sparsification: algorithms to select the best landmarks for localization [10, 11] and keyframe selection to sparsify the pose graph. Useful for processing large-scale maps or for lifelong mapping.

Dense reconstruction: a collection of dense reconstruction, depth fusion and surface reconstruction [38] algorithms. Also includes an interface to CMVS/PMVS2 [39]. See Section IV-E for details.

F. ROVIOLI: online VIO and localization frontend

ROVIOLI (ROVIO with Localization Integration) is maplab's mapping and localization frontend which is used to build maps from raw visual and inertial data and also localize w.r.t. existing maps online. It is built around the visual-inertial odometry framework ROVIO [3] and extends it with localization and mapping capabilities. The following two modes of operation are available: (i) *VIO mode (Visual Inertial Odometry)* in which a map is built based on the VIO estimates and (ii) *LOC mode* where additionally localization constraints are processed to track a (drift-free) global pose estimate w.r.t. a given map. The localization maps are either created directly in a previous (single-session) of ROVIOLI or are exported from the maplab-console. The preparation of a localization map within the console allows for building complex processing pipelines (e.g. multi-session maps, data selection and compression).

An overview of the (main) data flows and modules within ROVIOLI are shown in Fig. 4. The *Feature Tracking* module

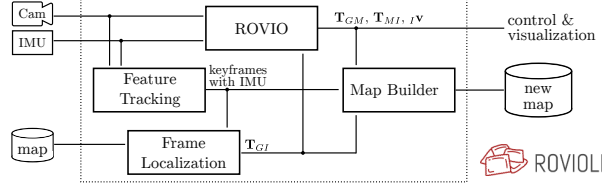


Fig. 4: Modules and data flows within ROVIOLI (ROVIO [3] with Localization Integration).

detects and tracks BRISK [34] or FREAK [35] keypoints. Feature correspondences between frames are established by matching descriptors from frame to frame. The expected matching window is predicted based on integrated gyroscope measurements to increase the efficiency and robustness. In *LOC mode*, keyframes containing feature points and descriptors are processed by the *Frame Localization* module to establish 2d-3d matches against the provided localization map. These 2d-3d matches are used to obtain a global pose estimate T_{GI_k} w.r.t. the map's frame of reference (see Fig. 3) using a P3P algorithm within a RANSAC scheme. The raw global pose estimates are fed to ROVIO where they are fused with the odometry constraints to estimate a transformation T_{GM} in addition to the local odometry pose T_{MI} . The outputs of all modules are synchronized within the *Map Builder* to construct a visual-inertial map (VI-map). The resulting map can serve as a localization map in subsequent sessions or can be loaded into the maplab console for further processing.

A process-internal publisher-subscriber data exchange layer manages the data flows between all modules within ROVIOLI. This architecture makes it easy to extend the current online pipeline with new algorithms, e.g. for on-line multiagent mapping, semantic SLAM, or localization research.

IV. USE-CASES

This section gives an overview of five common use cases of maplab: online mapping and localization, multi-session mapping, map maintenance, large-scale mapping and dense reconstruction. While maplab offers much more than that, we believe these examples highlight the capabilities of the system, the expected performance and its scalability.

Furthermore, we provide the related console commands to reproduce every example. The intention is to show that the following results can be obtained by relying solely on the user interface, without any additional code development. For more documentation, updated commands, datasets and tutorials, please visit our wiki page: www.github.com/ethz-asl/maplab/wiki.

A. Online mapping and localization with ROVIOLI

For many robotic applications it is of high importance to have access to (drift-free) global pose estimates. Such capability enables, e.g., teach&repeat scenarios, robotic manipulation and precise navigation. Within maplab, as a first step, we use ROVIOLI to create an initial VI-map of the desired area of operation. The sensor data can be provided

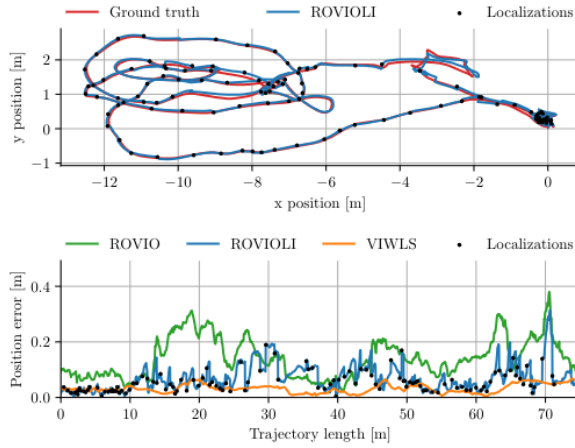


Fig. 5: Evaluation of ROVIOLI on the EuRoC machine hall dataset [40]. *Top*: Ground-truth positions overlaid with the ROVIOLI position estimates. *Bottom*: Position error of the visual-inertial odometry pipeline ROVIOLI [3], ROVIOLI and the optimized VI-map (VIWLS) compared to the ground truth.

either offline in a Rosbag or online using ROS topics. Upon completion, the VI-map is automatically loop-closed, optimized and optionally keyframed and summarized to obtain a compact localization map. In a second session the localization map can be passed to ROVIOLI to obtain drift-free global pose estimates in the mapped area.

We evaluated the ROVIOLI estimates against plain ROVIO [3] results and the estimates from a full-batch optimization on the EuRoC datasets [40]. To that end, in a first step, we created a localization map using one of the datasets. Then in a second step we processed a second EuRoC dataset using both ROVIOLI (using the previously built map) and ROVIO. The results are presented in Fig. 5 and Table I, where we compare the groundtruth error of ROVIO, ROVIOLI, and the full-batch optimized trajectory. These experiments demonstrate the drift-free performance of the system and the improvements upon the regular VIO estimation. Additionally, Table II shows timing information of ROVIO and ROVIOLI compared to ORB-SLAM2 [25].

B. Multi-session mapping

In many mapping applications, it is not possible to cover the entire environment within a single mapping session. Apart from that, it might be desirable to capture the environment in as many differing visual appearance conditions as possible [7]. Therefore, maplab offers tools to co-register maps from multiple sessions together and jointly refine them to obtain a single, consistent map.

Hence this use case demonstrates the process of creating a map of a university building from 4 individual trajectories. Each trajectory passes through the ground floor, staircases and one other floor of a building. Combined, they cover over 1,000 meters and contain about 463,000 landmarks. On such large maps, many of the common operations such as optimization or loop closure quickly become intractable without a careful selection of the data. For this reason,

TABLE I: Global position and orientation RMSEs on EuRoC datasets [40] for ROVIO (only VIO), ROVIOLI using one of the datasets as a localization map and ROVIO+VIWLS that corresponds to a full batch visual-inertial least-squares optimization (VIWLS). Additionally, the results of ORB-SLAM2 [25] (in batch and real-time) are compared. ROVIO and ROVIOLI use a single camera and IMU data whereas ORB-SLAM2 uses a stereo camera. The localization map for ORB-SLAM2 has been built in SLAM mode whereas the localization evaluation has been performed in localization mode. For V2-medium, we were unable to build a map with ORB-SLAM2's real-time mode as the estimator diverged (marked with X).

	MH1		V2-easy	
	*LOC: MH2	*LOC: V2-medium	position	orientation
ROVIO	0.178 m	1.49 deg	0.064 m	0.90 deg
ROVIOLI*	0.082 m	1.43 deg	0.057 m	1.57 deg
ROVIO+VIWLS	0.036 m	1.29 deg	0.027 m	1.06 deg
ORB-SLAM2* (batch mode)	0.084 m	0.78 deg	0.121 m	1.14 deg
ORB-SLAM2* (real-time)	0.464 m	13.34 deg	X	X

TABLE II: (a) Timing and CPU load for ROVIO, ROVIOLI and ORB-SLAM2 on EuRoC MH1 dataset processed at 20 Hz. In case of ROVIOLI and ORB-SLAM2 (marked with *), the estimator was set to localize against a map built from EuRoC MH2. All reported values have been measured on an Intel Xeon E3-1505M@2.8Ghz. A CPU load of 800% corresponds to fully utilizing all 8 (logical) cores of the CPU. (b) Single frame processing times for the individual blocks of ROVIOLI. The total time does not correspond to the sum of the individual blocks as they run in parallel. Instead, it is the time it takes for a single frame to be fully processed.

(a)			(b)	
	Frame update	CPU load	ROVIOLI frame update	
ROVIO	23 ms	56%±7.7%	ROVIO update	22.7 ms
ROVIOLI*	44 ms	105%±14.8%	Feature tracking	20.6 ms
ORB-SLAM2* (batch mode)	63 ms	162%±10.9%	Localization	20.4 ms
			Map building	3.2 ms
			Total	44.2 ms

we employ a keyframing scheme using heuristics based on vertex distance, orientation, and landmark covisibility. The loop closure algorithm of maplab correctly identifies the geometric transformations between all missions and the non-linear optimization refines the geometry. The result is a compact, geometrically-consistent localization map of 8.2 MB ready to be used by ROVIOLI for localization within the entire building as shown in Fig. 1.

This use case can be reproduced using the following commands in the maplab console:

```
# Load multiple single session maps from ROVIOLI.
load_merge_all_maps --maps_folder YOUR_MAPS_FOLDER
# Keyframing and initial optimization.
kfh
optvi
# Set one mission as base, anchor the others.
set_mission_baseframe_to_known
anchor_all_missions
# Pose-graph relaxation, loop-closure, optimization.
relax
lc
optvi
visualize
```


C. Map maintenance

Large feature-based models, potentially built in multiple sessions, easily comprise thousands of landmarks and reach considerable storage size. However, it is not really necessary to keep all of the landmarks to guarantee good localization quality with ROVIOLI. Maplab offers a map summarization functionality based on [11] that uses an integer-based optimization to perform the landmark selection. The algorithm attempts to remove the least commonly seen landmarks but at the same time maintain a balanced coverage of the environment. Maplab also includes a keyframing algorithm to remove redundant vertices and only keep the ones necessary for an efficient and accurate state estimation. By removing the vertices we also eliminate many vertex-landmark associations that contain descriptors of considerable size. Both summarization and keyframing permit to significantly reduce the model size without a large loss in pose estimation quality.

The map maintenance is demonstrated on a database map built from 4 mapping sessions recorded on the ground floor of the building introduced in Section IV-B. Each mapping session covers about 90 meters and contains about 20,000 landmarks, out of which about 5,000 are considered reliable. A 5th dataset is used as a query – we try to localize each vertex against the database, built from the 4 datasets, and verify if the position error is smaller than 50 cm. We compare the recall of localization maps that were pre-processed in different ways, either summarized, keyframed or both.

Fig. 6 presents the influence of landmark summarization and keyframing on the localization map size and demonstrates how those approaches affect the localization. The results confirm that keyframing significantly reduces the localization map size with a rather marginal loss of localization quality. Similarly, summarization can reduce the total amount of landmarks by 90% without grave consequences. When these methods are combined we can reduce the map size 13 times and keep the recall level at 51%, compared to 60% for the full map.

```
# Keyframe the map and sparsify landmarks to 10,000.
kfh
landmark_sparsify --num_landmarks_to_keep=10000
```

D. Large-scale mapping

In this use-case we would like to demonstrate the large-scale mapping capabilities of maplab and the applicability to a sensor other than the VI-sensor [41]. To that end we used the publicly available Google Tango tablets, and recorded a large-scale, multi-session map of the old town of Zurich. We exported the raw visual-inertial data and processed it with ROVIOLI to obtain the initial open loop maps. We then loaded these maps into the maplab console for alignment and optimization and applied the same tools as described in Section IV-B. The bundle adjustment and pose-graph relaxation was performed on a desktop computer with 32 GB RAM overnight. An orthographic projection of the optimized VI-map onto the map of Zurich, as well as further details about the map can be found in Fig. 7. The figure shows that

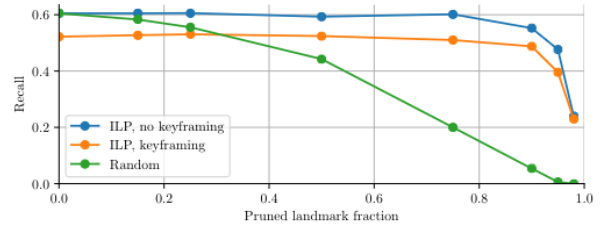


Fig. 6: The localization performance and map size after ILP landmark summarization and keyframing+summarization (in brackets). Keyframing removes vertices including vertex-landmark associations, effectively making the map smaller. The original map had 6,258 vertices whereas the keyframed map contains 760. Keyframing consistently reduces the recall by a few percent while summarization only affects the quality when the pruned landmark fraction exceeds 85%. For comparison, we provide a recall curve for a random selection of landmarks to be removed.

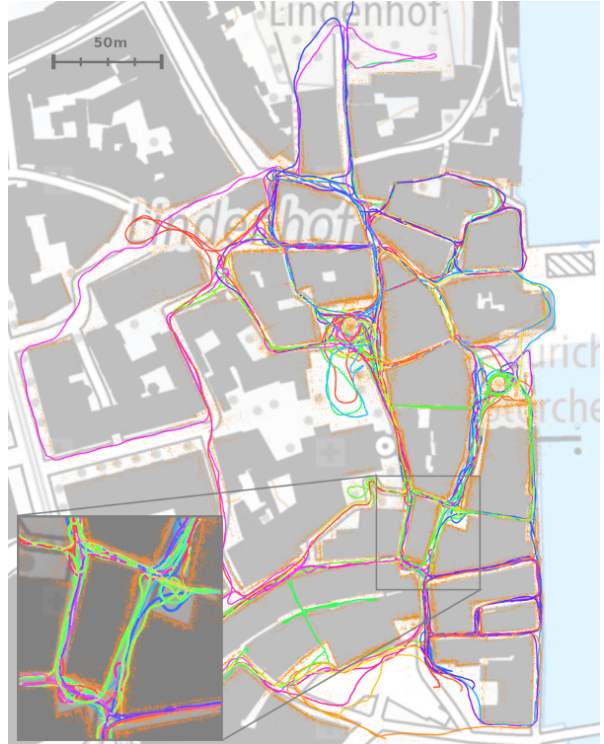


Fig. 7: Large-scale, multi-session VI-map of Zurich's old town. Built from the raw visual-inertial data recorded in 45 sessions using Google Tango tablets on two different days (sunny and cloudy). The total duration of the recordings is 231 min. The final map contains trajectories with a total length of 16.48 km, 435k landmarks with 7.3M observations and has a size of 480 MB. The map is available on the maplab wiki page for download.

the resulting map is consistent with the building and streets across most of the map with some minor inconsistencies in areas of low coverage.

E. Dense reconstruction

Many applications in robotics, such as path planning, inspection and object detection require a more dense 3d representation of the environment. Maplab offers several dense reconstruction tools, which use the optimized *vertex* poses of the sparse map to compute dense depth information based on camera images attached to the VI-map.

1) *Stereo dense reconstruction*: In order to compute depth maps from multi-camera systems, this tool first identifies stereo cameras that are suitable for planar rectification. It then utilizes a (semi-global) block matcher to compute depth maps for every stereo pair along the trajectory. The resulting depth maps (or point clouds) are attached to the VI-map and stored in the resource system. The following commands assume that the maps are already aligned, loop-closed and optimized as described in Section IV-B.

```
stereo_dense_reconstruction
```

2) *TSDF-based depth fusion*: Once the VI-map contains depth information, e.g. obtained using the above described commands or an RGB-D sensor, the globally consistent camera poses of the VI-map can be utilized to create an equally consistent global 3d reconstruction. To that end, maplab employs voxblox [38], a volumetric mapping library, for TSDF-based depth fusion and surface reconstruction. The following commands will insert depth maps or point cloud data into a voxblox grid and store a surface mesh to the file system. The top row of Fig. 8 shows the reconstruction results of 3 combined EuRoC machine hall datasets [40].

```
create_tsdf_from_depth_resource
--dense_tsdf_voxel_size_m 0.10
--dense_tsdf_truncation_distance_m 0.30
export_tsdf
--dense_result_mesh_output_file YOUR_FILE
```

3) *Export to CMVS/PMVS2*: For more accurate dense reconstructions maplab offers an export command to convert the sparse VI-map and images to the input data format for the open-source multi-view-stereo pipeline, CMVS/PMVS2 [39]. Even though the export of grayscale images is supported, the best results are obtained using RGB images. The VI-map and the resulting 3D reconstruction can be seen in the bottom row of Fig. 8.

```
export_for_pmvs
--pmvs_reconstruction_folder EXPORT_FOLDER
```

V. USING MAPLAB FOR RESEARCH

All the algorithms and console commands required for the use-cases in Section IV are available in maplab and constitute most of the basic tools needed in visual-inertial mapping and localization. Furthermore, a rich set of helper functions, queries, and manipulation tools are provided to ease rapid prototyping of new algorithms. The plugin architecture of the console allows for an easy integration of new algorithms

into the system. Examples demonstrating how to extend the framework are provided in the project's wiki pages. We would like to invite the community to take advantage of this research-friendly design.

VI. CONCLUSIONS

This work presents maplab, an open framework for visual-inertial mapping and localization with the goal of making research in this field more efficient by providing a collection of basic algorithms and letting researchers focus on actual tasks. All components in maplab are written in a flexible and extensible way such that novel algorithms that rely on visual-inertial state estimates or localization can be integrated and tested easily. For this reason, the framework provides an implementation of the most important tools required in mapping and localization related research such as visual-inertial optimization, a loop-closure/localization backend, multi-session map merging, pose-graph relaxation and extensive introspection and visualization tools. All these algorithms are made accessible from a console-based user interface where they can be applied to single or multi-session maps. Such a workflow has proven to be very efficient when prototyping new algorithms or tuning parameters.

Secondly, the framework contains an online visual-inertial mapping and localization front-end, named ROVIOLI. It can build new maps from raw visual and inertial sensor data and additionally track a global (drift-free) pose in real-time if a localization map is provided. Previous work made use of this capability on different robotic platforms and demonstrated its ability of accurately tracking a global pose for a multitude of applications, including navigation and trajectory following.

VII. ACKNOWLEDGEMENT

We would like to acknowledge the many other contributors of maplab, most importantly: Titus Cieslewski, Mathias Bürki, Timo Hinzmann, Mathias Gehrig and Nicolas Degen. Furthermore we would also like to thank Michael Blösch the author of ROVIO. The research leading to these results has received funding from Google Tango and the EU H2020 project UP-Drive under grant no. 688652.

REFERENCES

- [1] S. Leutenegger, S. Lynen, M. Bosse, R. Siegwart, and P. Furgale, "Keyframe-based visual-inertial odometry using nonlinear optimization," *IJRR*, 2015.
- [2] N. Snavely, S. M. Seitz, and R. Szeliski, "Photo tourism: exploring photo collections in 3d," in *SIGGRAPH*, 2006.
- [3] M. Bloesch, S. Omari, M. Hutter, and R. Siegwart, "Robust visual inertial odometry using a direct ekf-based approach," in *IROS*, 2015.
- [4] M. Burri, H. Oleynikova, M. Achtelik, and R. Siegwart, "Real-time visual-inertial mapping, re-localization and planning on-board MAVs in unknown environments," in *IROS*, 2015.
- [5] T. Hinzmann, T. Schneider, M. Dymczyk, A. Schaffner, S. Lynen, R. Siegwart, and I. Gilitschenski, "Monocular visual-inertial SLAM for fixed-wing UAVs using sliding window based nonlinear optimization," in *ISVC*, 2016.
- [6] T. Hinzmann, T. Schneider, M. Dymczyk, A. Melzer, T. Manter, R. Siegwart, and I. Gilitschenski, "Robust map generation for fixed-wing UAVs with low-cost highly-oblique monocular cameras," in *IROS*, 2016.

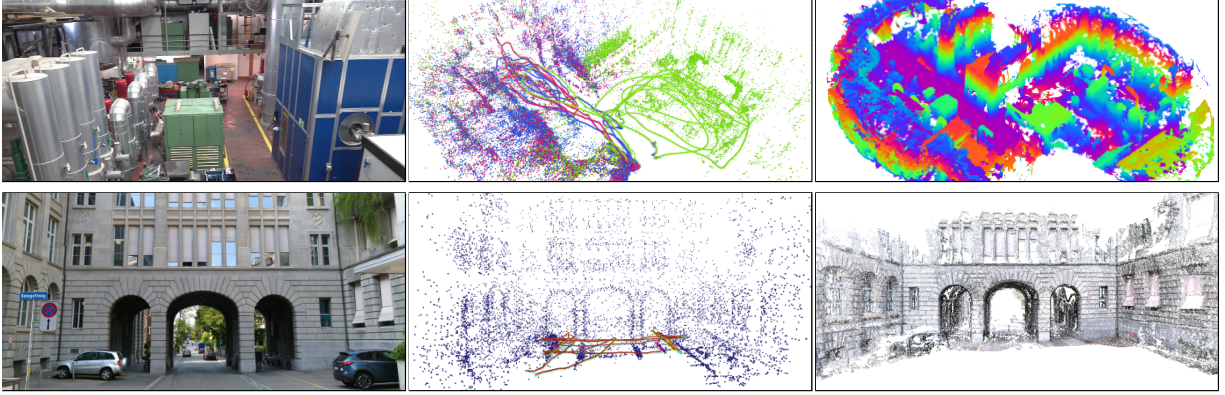


Fig. 8: Two different dense reconstruction tools are available in maplab. *Top*: stereo dense reconstruction is used to compute depth maps based on grayscale images and optimized camera poses. They are then fused in voxblox [38] to create a surface mesh. 3 EuRoC datasets [4] (MH1-3) are combined to create an aligned and optimized VI-map. *Bottom*: CMVS/PMVS2 [39] reconstruction results based on a single recording session using a multi-camera system with a RGB camera.

- [7] M. Buerki, I. Gilitschenski, E. Stumm, R. Siegwart, and J. Nieto, “Appearance-based landmark selection for efficient long-term visual localization,” in *IROS*, 2015.
- [8] L. Trafflet, T. Eppenberger, A. Millane, T. Schneider, and R. Siegwart, “Target-based calibration of underwater camera housing parameters,” in *SSRR*, 2016.
- [9] P. Fankhauser, M. Bloesch, P. Krüsi, R. Diethelm, M. Wermelinger, T. Schneider, M. Dymczyk, M. Hutter, and R. Siegwart, “Collaborative navigation for flying and walking robots,” in *IROS*, 2016.
- [10] M. Dymczyk, S. Lynen, T. Cieslewski, M. Bosse, R. Siegwart, and P. Furgale, “The gist of maps-summarizing experience for lifelong localization,” in *ICRA*, 2015.
- [11] M. Dymczyk, S. Lynen, M. Bosse, and R. Siegwart, “Keep it brief: Scalable creation of compressed localization maps,” in *IROS*, 2015.
- [12] M. Dymczyk, I. Gilitschenski, R. Siegwart, and E. Stumm, “Map summarization for tractable lifelong mapping,” in *RSS Workshop*, 2016.
- [13] M. Dymczyk, T. Schneider, I. Gilitschenski, R. Siegwart, and E. Stumm, “Erasing bad memories: Agent-side summarization for long-term mapping,” in *IROS*, 2016.
- [14] H. Merzić, E. Stumm, M. Dymczyk, R. Siegwart, and I. Gilitschenski, “Map quality evaluation for visual localization,” in *ICRA*, 2017.
- [15] M. Fehr, M. Dymczyk, S. Lynen, and R. Siegwart, “Reshaping our model of the world over time,” in *ICRA*, 2016.
- [16] F. Blöchliger, M. Fehr, M. Dymczyk, T. Schneider, and R. Siegwart, “Topomap: Topological mapping and navigation based on visual slam maps,” *arXiv: 1709.05533*, 2017.
- [17] M. Gehrig, E. Stumm, T. Hinzmann, and R. Siegwart, “Visual place recognition with probabilistic voting,” in *ICRA*, 2017.
- [18] H. Oleynikova, M. Burri, S. Lynen, and R. Siegwart, “Real-time visual-inertial localization for aerial and ground robots,” in *IROS*, 2015.
- [19] S. Lynen, M. Bosse, P. Furgale, and R. Siegwart, “Placeless place-recognition,” in *3DV*, 2014.
- [20] T. Cieslewski, S. Lynen, M. Dymczyk, S. Magnenat, and R. Siegwart, “Map api-scalable decentralized map building for robots,” in *ICRA*, 2015.
- [21] G. Klein and D. Murray, “Parallel tracking and mapping for small ar workspaces,” in *ISMAR*, 2007.
- [22] C. Forster, M. Pizzoli, and D. Scaramuzza, “Svo: Fast semi-direct monocular visual odometry,” in *ICRA*, 2014.
- [23] J. Engel, T. Schöps, and D. Cremers, “LSD-SLAM: Large-scale direct monocular slam,” in *ECCV*, 2014.
- [24] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardos, “ORB-SLAM: a versatile and accurate monocular SLAM system,” *T-RO*, 2015.
- [25] R. Mur-Artal and J. D. Tardos, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *T-RO*, 2017.
- [26] M. Bosse, P. Newman, J. Leonard, and S. Teller, “Simultaneous Localization and Map Building in Large-Scale Cyclic Environments Using the Atlas Framework,” *IJRR*, 2004.
- [27] J. McDonald, M. Kaess, C. Cadena, J. Neira, and J. Leonard, “Real-time 6-DOF multi-session visual SLAM over large-scale environments,” *RAS*, 2013.
- [28] W. Churchill and P. Newman, “Experience-based navigation for long-term localisation,” *IJRR*, 2013.
- [29] C. Sweeney, “Theia multiview geometry library: Tutorial & reference,” <http://theia-sfm.org>.
- [30] P. Moulon, P. Monasse, R. Perrot, and R. Marlet, “Openmv: Open multiple view geometry,” in *RRPR*, 2016.
- [31] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, “Ros: an open-source robot operating system,” in *ICRA*, 2009.
- [32] G. Guennebaud, B. Jacob, *et al.*, “Eigen v3,” <http://eigen.tuxfamily.org>, 2010.
- [33] S. Agarwal and K. Mierle, “Ceres solver,” <http://ceres-solver.org>.
- [34] S. Leutenegger, M. Chli, and R. Y. Siegwart, “Brisk: Binary robust invariant scalable keypoints,” in *ICCV*, 2011.
- [35] A. Alahi, R. Ortiz, and P. Vandergheynst, “Freak: Fast retina keypoint,” in *CVPR*, 2012.
- [36] S. Lynen, T. Sattler, M. Bosse, J. A. Hesch, M. Pollefeys, and R. Siegwart, “Get out of my lab: Large-scale, real-time visual-inertial localization,” in *RSS*, 2015.
- [37] N. Sünderhauf and P. Protzel, “Switchable constraints for robust pose graph SLAM,” in *IROS*, 2012.
- [38] H. Oleynikova, Z. Taylor, M. Fehr, R. Siegwart, and J. Nieto, “Voxblox: Incremental 3d euclidean signed distance fields for on-board MAV planning,” *arXiv:1611.03631*, 2016.
- [39] Y. Furukawa, B. Curless, S. M. Seitz, and R. Szeliski, “Towards internet-scale multi-view stereo,” in *CVPR*, 2010.
- [40] M. Burri, J. Nikolic, P. Gohl, T. Schneider, J. Rehder, S. Omari, M. W. Achtelik, and R. Siegwart, “The EuRoC micro aerial vehicle datasets,” *IJRR*, 2016.
- [41] J. Nikolic, J. Rehder, M. Burri, P. Gohl, S. Leutenegger, P. Furgale, and R. Siegwart, “A synchronized visual-inertial sensor system with fpga pre-processing for accurate real-time slam,” in *ICRA*, 2014.